ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Paul TAPPONNIER**
- **Chinese Name: 王安宇**
- **Email: paul.tapponnier@utt.fr**
- **Student ID: X1150038**
- **University: NTHU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer: 1.

There are 594 documents in the 'sci.med' category.

2. Frequent pattern mining only selects information based on the proportion of documents that contain a given set of words, called support. If the proportion is above the threshold, then it is kept. Isolating specific categories allows us to increase this proportion so that data are not diluted by other categories, allowing us to gather proper information related to the themes selected.

AI used for this answer: No

If yes, how:

External references (links):

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: About 900 documents would be instantly lost. The support would be less relevant because of the scarcity of documents . Losing 40% of our data for a missing label is not a good way to approach data mining. An alternative could be to just estimate the missing values based on the other 60% documents with a category_name label. Or, we could just keep them for other purposes, like clustering who doesn't necessarily need labels.

AI used for this answer: No

If yes, how:

External references (links):

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. No, they don't perfectly match the original but they are really close since the biggest gap concerns comp.graphics (+1.9pt). The order change as well as soc.religion.christian is 1st in the original but 3rd in the sample.

Category	**Original**	**% original**	sample	% sample	gap
soc.religion.christian	**599**	**26,5 %**	**248**	**24,8 %**	**−1,7 pt**
sci.med	**594**	**26,3 %**	**260**	**26,0 %**	**−0,3 pt**
comp.graphics	**584**	**25,9 %**	**278**	**27,8 %**	**+1,9 pt**
alt.atheism	**480**	**21,3 %**	**214**	**21,4 %**	**+0,1 pt**

2. There, all the categories are nearly in the same proportions, but if a category made up only 2% of the entire corpus, there is a high risk the sampling would erase those rare instances and the category would be under-represented or even missing. Stratified sampling is a good way to respect the proportions of each category.

AI used for this answer: No

If yes, how:

External references (links):

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:
1. The total number of features increased from 35482 (stop_words='english') to 35788 (stop_words=None), so 306 more.
2. The sparsity slightly decreases since we increase the number of non-zero entries since stop words appear in nearly all documents. The effect is nearly negligible as it is only 300 mostly full columns over 35 500 columns. Their effect, though, could be much more impaxtful as thse columns are dense, full!

AI used for this answer: No

If yes, how:

External references (links):

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?

2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold	**Number of terms kept (keep == True)**
0.0	35,482
0.01	5,089
0.05	1,491

1.

Vocabulary shrinks sharply with an overall -96% reduction with a threshold of 0.05 (versus -86% with 0.01). At 0, nothing is removed, which means that no term is perfectly constant.

2. Words whose variance are close to zero are words that have a low document frequency, which means that these words appear in only a few documents out of the 2257 ones. For example, we got

35467	**zipped**	**0.000443**	**False**
35468	**zimmerman**	**0.000443**	**False**
35469	**4202**	**0.000443**	**False**

Which have a really low variance since they simply do not appear much in the documents and so have a column of almost all zeros.

3. Near-zero variance words are words that do not tell us a lot of useful information and could even lead to noise that could not be generalized in our model, which is called overfitting.
So, our model will not lose any sense while also reducing dimensionality and calculation time.

AI used for this answer: No

If yes, how:

External references (links):

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

RANK	PEARSON	R	SPEARMAN	R
1	**thanks**	**+0.28**	**graphics**	**+0.52**
2	**writes**	**−0.23**	**people**	**−0.32**
3	**keywords**	**+0.23**	**files**	**+0.32**
4	**looking**	**+0.21**	**image**	**+0.31**
5	**vga**	**+0.21**	**god**	**−0.31**
6	**video**	**+0.21**	**3d**	**+0.31**
7	**people**	**−0.21**	**program**	**+0.29**

8	**card**	**+0.21**	**thanks**	**+0.29**
9	**hi**	**+0.20**	**file**	**+0.28**
10	**host**	**+0.20**	**windows**	**+0.28**

1. For Spearman, mainly domain specific terminology ("graphics", "3d", "image" etc.) while Pearson have more generic words such as "hi", "thanks", "looking", but still have domain-specific terminology. Negative r (people, god, writes) are for other categories.

2. Pearson measures how well the values follow a straight line. Works with raw counts. Extremely sensitive to extreme values.
Spearman, on the other hand, measures how well the rankings move in the same direction, thus making it robust to extreme values. It measures a monotonic relation.

3. A word would have a high Spearman correlation but a significantly lower Pearson correlation when its relationship with a topic is strongly monotonic but not linear. Happens when, per example, a word is regularly tied to the topic but with unequals counts (1 or 2 in most documents but a dozen in a few ones).

AI used for this answer: Yes

If yes, how: Understanding of the differences between both features correlation methods.

External references (links): https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html and https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup: Lowering minSup from 9 to 3 made MaxFPGrowth find far more patterns (710 → 9,547, about 13×) and run much slower (0.08 s → 1.94 s, about 23×). A lower threshold lets many rare word combinations count as frequent, so the algorithm has more candidates to explore.

Top-K FAE always returned exactly k patterns (100 or 500), with runtime rising only slightly (0.05 s → 0.17 s).

Answer (2) — Top-K k vs MaxFPGrowth minSup: minSup sets a fixed frequency threshold, so the number of patterns returned depends on the data. k fixes the output size instead: FAE returns exactly the k most frequent patterns.

Answer (3) — Maximal vs standard frequent patterns: A maximal pattern has no frequent superset. Every subset of a frequent pattern is also frequent, so keeping only the largest patterns removes all that redundancy.

AI used for this answer: No

If yes, how:

External references (links):

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:
1. Each document has ~35,000 coordinates (one per word), but a scatter plot only has 2 or 3 axes. Plotting two raw word columns would show almost nothing, since most values are zero.
2. Dimensionality reduction algorithms (PCA, t-SNE, UMAP).

AI used for this answer: No

If yes, how:

External references (links):

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

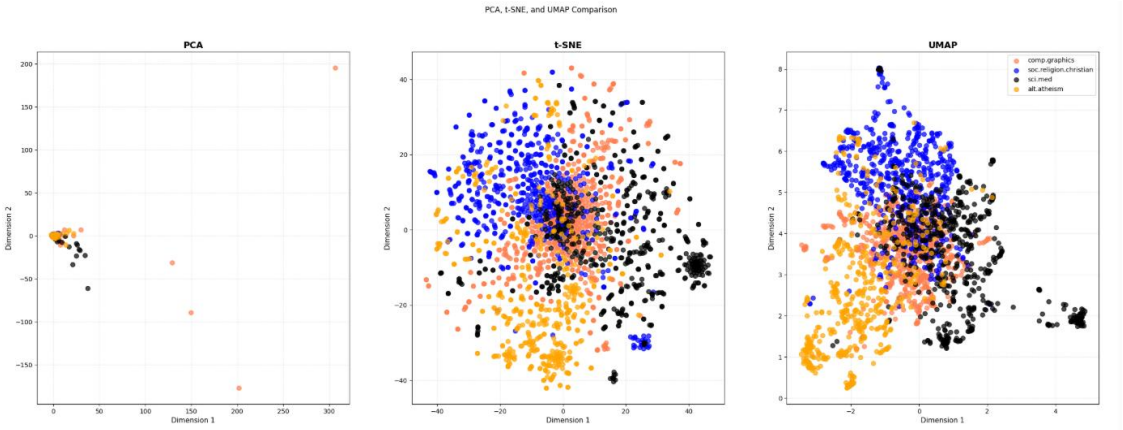
Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP)** shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:



1. t-SNE and UMAP show the clearest category regions: soc.religion.christian (blue) at the top, alt.atheism (orange) at the bottom/left, sci.med (black) on the right with small isolated islands, comp.graphics in the middle. The categories still overlap in the center. PCA shows heavy overlap: almost all documents are squashed into one blob near the origin, while a few comp.graphics outliers stretch the axes.

2. PCA is a linear projection that keeps the directions of largest variance. In raw word counts, these directions are driven by a few very long documents (the outliers in the plot), not by topic. t-SNE and UMAP are non-linear and preserve each document's nearest neighbours, so documents with similar vocabulary end up grouped together.

3. t-SNE only preserves local neighbourhoods. The distances between clusters, and their sizes, are not meaningful and can change with parameters or random initialization. Two clusters far apart in the plot are not necessarily very different.

AI used for this answer: Yes

If yes, how: troubleshooting a UTF-8 encoding error in the notebook, concept explanation, and grammar review of my draft.

- **External references (links):** https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html
- https://distill.pub/2016/misread-tsne/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. 4 binary columns, one per category.
2. It assumes the categories are ordered and evenly spaced (0 < 1 < 2 < 3), e.g. that alt.atheism (0) is closer to comp.graphics (1) than to soc.religion.christian (3).
3. Each category gets its own dimension, so there's no order and all categories are equally distant from each other.

AI used for this answer: Yes

If yes, how: Resuming the notions implied

External references (links): https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features

---